

Módulo 4 — Desarrollo de Aplicaciones con IA

TributIA

Evaluación Semana 5 — Observabilidad, rendimiento y escalabilidad

Equipo	11
Integrantes	Daniel Enrique Zaldaña Castillo · Edwin Adony Ulloa Díaz · Anthony Enrique Alvarenga Mayen
Docente	Ing. Marco Arévalo Zambrano
Fecha	15 de agosto de 2026
Repositorio	https://github.com/DanCastSV/TributIA

1. Descripción del servicio y flujo crítico seleccionado

TributIA es una plataforma web Django que analiza documentos tributarios (facturas, constancias, comprobantes en PDF/PNG/JPG) mediante un pipeline de OCR + spaCy + Google Gemini, expuesto tanto por la interfaz web como por una API interna versionada (`/api/v1/`).

Flujo crítico elegido: `POST /api/v1/analizar-documento/` — el pipeline completo de análisis (extracción de texto → regex → spaCy → Gemini → persistencia). Es el flujo que hace todo el trabajo pesado del sistema y el más propenso a errores, timeouts y variabilidad de latencia, al depender de Tesseract/OCR, spaCy y una llamada externa a la API de Gemini.

Para la línea base de rendimiento (sección 7) se usa además `GET /api/v1/health/` — no porque sea el flujo crítico, sino porque permite correr las ≥ 20 solicitudes que pide la rúbrica sin consumir cuota real y limitada de Gemini. El endpoint de análisis se mide aparte con una muestra real más pequeña (justificado en la sección 7).

2. Preguntas de observabilidad y campos registrados

- ¿Cuánto tarda cada etapa del pipeline (extracción de texto, spaCy, Gemini) por separado, no solo el total?
- ¿Qué proporción de solicitudes al endpoint principal terminan en error, y de qué tipo?
- ¿Con qué versión/modelo de Gemini se generó un análisis dado?
- ¿Se puede rastrear una solicitud específica de principio a fin?
- ¿Cuál es la latencia normal del servicio bajo carga baja, como referencia para notar degradación después?

Campos registrados por cada request HTTP (`core/middleware.py` , logger `tributia.requests`):

Campo	Descripción
<code>request_id</code>	UUID único por solicitud, también devuelto en el header <code>X-Request-ID</code>
<code>metodo</code>	Método HTTP
<code>ruta</code>	Path solicitado
<code>status</code>	Código de respuesta HTTP
<code>duracion_ms</code>	Duración total de la request

Campos registrados por cada etapa del pipeline de IA (`core/services/analizador.py` , logger `core.services.analizador`), correlacionados automáticamente con el mismo `request_id` :

Campo	Descripción
documento_id	ID interno del documento (no el nombre de archivo ni su contenido)
etapa	extraccion_texto , spacy o gemini
duracion_ms	Duración de esa etapa específica
modelo	Nombre del modelo Gemini usado (etapa gemini)
caracteres_extraidos	Cantidad de caracteres extraídos (nunca el texto en sí)
tipo_error	archivo_invalido , no_autenticado , documento_no_tributario , o el nombre de la excepción Python
confianza_clasificacion	Score 0.0–1.0 calculado por calcular_confianza()

3. Código de instrumentación

Correlación por request_id (`core/logging_context.py`): usa `contextvars` para que cualquier log emitido durante una request quede automáticamente etiquetado con el mismo `request_id`, sin pasarlo por parámetro entre funciones.

Formato JSON (`core/logging_utils.py`): un `Formatter` que serializa cada línea como JSON, tomando cualquier campo pasado vía `extra={ ... }`.

Middleware (`core/middleware.py`): mide la duración total de cada request y registra `request_completada` al finalizar.

```
class RequestLoggingMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        request_id = str(uuid.uuid4())
        token = request_id_var.set(request_id)
        request.request_id = request_id
        inicio = time.monotonic()

        try:
            response = self.get_response(request)
            duracion_ms = round((time.monotonic() - inicio) * 1000, 1)
            response["X-Request-ID"] = request_id

            nivel = logging.INFO if response.status_code < 500 else logging.ERROR
            logger.log(nivel, "request_completada", extra={
                "request_id": request_id,
                "metodo": request.method,
                "ruta": request.path,
                "status": response.status_code,
                "duracion_ms": duracion_ms,
            })
            return response
        finally:
            request_id_var.reset(token)
```

LOGGING en settings.py : reemplaza la configuración implícita de Django, que por defecto solo activa el handler de consola cuando `DEBUG=True` — ese fue exactamente el motivo por el que, en Semana 4, un error real (`InvalidStorageError`) no dejó ningún rastro dentro del contenedor (`DEBUG=False`). Ahora los logs siempre van a `stdout` en JSON, sin importar `DEBUG`.

Pipeline instrumentado (`core/services/analizador.py`): cada etapa se cronometra por separado:

```

inicio = time.monotonic()
 analisis_gemini = analizar_documento_con_gemini(texto, { ... })
 logger.info("etapa_completada", extra={
     "documento_id": documento_id,
     "etapa": "gemini",
     "modelo": MODEL_NAME,
     "duracion_ms": _duracion_ms(inicio),
 })

```

Evidencia de una solicitud exitosa

Login por curl + subida de una factura PDF real a `POST /api/v1/analizar_documento/` :

```

$ curl -s -b cookies.txt -X POST \
  -F "archivo=@media/documentos/ ... /factura.pdf" \
  -w "\nHTTP_STATUS:%{http_code}\n" \
  http://localhost:8000/api/v1/analizar_documento/
HTTP_STATUS:201
X-Request-ID: 00dae09b-13bc-4990-9e76-956e1e5cb3e5

```

Logs correlacionados por `request_id` (`docker compose logs web | grep 00dae09b`):

```

{"timestamp": "2026-08-15T10:33:14", "nivel": "INFO", "logger": "core.services.analizador",
"mensaje": "analisis_iniciado", "documento_id": 2, "request_id": "00dae09b- ... "}
{"timestamp": "2026-08-15T10:33:15", "nivel": "INFO", "logger": "core.services.analizador",
"mensaje": "etapa_completada", "etapa": "extraccion_texto", "duracion_ms": 67.3,
"caracteres_extraidos": 2129, "request_id": "00dae09b- ... "}
{"timestamp": "2026-08-15T10:33:15", "nivel": "INFO", "logger": "core.services.analizador",
"mensaje": "etapa_completada", "etapa": "spacy", "duracion_ms": 58.8, "request_id":
"00dae09b- ... "}
{"timestamp": "2026-08-15T10:33:17", "nivel": "INFO", "logger": "core.services.analizador",
"mensaje": "etapa_completada", "etapa": "gemini", "modelo": "models/gemini-2.5-flash-lite",
"duracion_ms": 2312.6, "request_id": "00dae09b- ... "}
{"timestamp": "2026-08-15T10:33:17", "nivel": "INFO", "logger": "core.services.analizador",
"mensaje": "analisis_completado", "duracion_total_ms": 2470.0, "confianza_clasificacion":
0.94, "request_id": "00dae09b- ... "}
{"timestamp": "2026-08-15T10:33:17", "nivel": "INFO", "logger": "tributia.requests",
"mensaje": "request_completada", "metodo": "POST", "ruta": "/api/v1/analizar_documento/",
"status": 201, "duracion_ms": 4043.7, "request_id": "00dae09b- ... "}

```

Interpretación: el `request_id` correlaciona los 6 eventos de esta única solicitud, desde el inicio del pipeline hasta la respuesta HTTP. La etapa `gemini` (2312.6 ms) domina el tiempo total del pipeline (2470.0 ms) — extracción de texto y spaCy juntas suman menos de 130 ms.

Evidencia de una entrada inválida / error controlado

Mismo flujo, enviando un archivo `.txt` (extensión no permitida):

```
$ curl -s -b cookies.txt -X POST -F "archivo=@prueba.txt" \
-w "\nHTTP_STATUS:%{http_code}\n" \
http://localhost:8000/api/v1/analizar-documento/
{"error": "archivo_invalido", "detalle": "Extensión no permitida. Use: pdf, png, jpg, jpeg"}
HTTP_STATUS:400
X-Request-ID: f716d205-ea3d-4f70-bf04-4e31c1f12110
```

```
{"timestamp": "2026-08-15T10:33:19", "nivel": "WARNING", "logger": "core.api_views",
"mensaje": "solicitud_rechazada", "tipo_error": "archivo_invalido", "request_id":
"f716d205- ... "}
{"timestamp": "2026-08-15T10:33:19", "nivel": "INFO", "logger": "tributia.requests",
"mensaje": "request_completada", "metodo": "POST", "ruta": "/api/v1/analizar-documento/",
"status": 400, "duracion_ms": 1830.1, "request_id": "f716d205- ... "}
```

Interpretación: el `tipo_error` (`archivo_invalido`) queda registrado explícitamente, sin inspeccionar el status por separado, correlacionado con el mismo `request_id` del header `X-Request-ID` devuelto al cliente.

4. Datos excluidos por privacidad/seguridad

Los logs de esta entrega **nunca incluyen**:

- Contenido del documento ni el texto extraído por OCR (solo su longitud en caracteres).
- Datos personales: NIT, DUI, nombre de empresa/cliente, dirección, correo, teléfono (viven en la base de datos, no en los logs).
- Credenciales o secretos: `GEMINI_API_KEY` , `SECRET_KEY` , contraseñas, cookies de sesión, tokens CSRF, encabezados `Authorization` .
- Cuerpo completo de la request/response (el middleware solo registra método, ruta, status y duración).
- Nombre real del archivo subido (podría contener información personal puesta por el usuario).

Lo que sí se loguea: identificadores internos (`documento_id` , entero autoincremental) y metadatos operativos (duración, etapa, tipo de error, modelo usado).

5. Escenario y resultados de la medición de rendimiento

Ambiente: contenedor Docker (`docker compose up -d --build`), `web` (Django + gunicorn, 3 workers) + `db` (PostgreSQL 16). Medido desde el equipo host (Windows) contra `http://localhost:8000` publicado por Docker Desktop.

Versión del código: commit `78a35c9` + los cambios de instrumentación de esta entrega, sobre `main` .

Componente IA: `gemini-2.5-flash-lite` (`core/ia/gemini_client.py::MODEL_NAME`).

Herramienta: `scripts/medir_rendimiento.py` (script propio, sin dependencias nuevas — `urllib` / `statistics` de la librería estándar).

Escenario A — línea base (20 solicitudes reales), `GET /api/v1/health/`

Se eligió `/health/` para las 20+ solicitudes en vez del endpoint de análisis porque este último consume cuota real y limitada de la API de Gemini; 20 llamadas reales a Gemini solo para medir rendimiento no es un uso justificable de esa cuota compartida.

`/health/` sí ejecuta trabajo real (consulta a PostgreSQL + verificación de Tesseract) dentro del mismo contenedor Docker.

```
$ python scripts/medir_rendimiento.py --url http://localhost:8000/api/v1/health/ --n 20 --
salida docs/medicion_health.json
[1/20] status=200 duracion_ms=86.1
[2/20] status=200 duracion_ms=11.8
...
[20/20] status=200 duracion_ms=11.4
```

≡ Resultado ≡

```
{
  "n": 20,
  "p50_ms": 12.1,
  "p95_ms": 35.6,
  "max_ms": 86.1,
  "min_ms": 10.6,
  "promedio_ms": 17.9,
  "tasa_error_pct": 0.0,
  "errores": 0
}
```

Interpretación: 0% de error en las 20 solicitudes. p50 de 12.1 ms y p95 de 35.6 ms — muy rápido, coherente con un endpoint liviano. El máximo (86.1 ms, primera solicitud) es un efecto de arranque en frío, no representativo del resto.

Escenario B — muestra real del flujo crítico, POST /api/v1/analizar-documento/

3 llamadas reales (no 20, por la razón de cuota explicada arriba) para caracterizar la latencia real del pipeline de IA:

#	extracción (ms)	spaCy (ms)	Gemini (ms)	Pipeline total (ms)	Request HTTP total (ms)	Status
1	67.3	58.8	2312.6	2470.0	4043.7 *	201
2	73.3	63.5	2180.7	2342.3	2396.5	201
3	89.9	49.9	1562.2	1715.3	1780.4	201

* Solicitud #1: primera tras docker compose up (arranque en frío, ver sección 6). Las solicitudes #2 y #3, con el contenedor ya "caliente", muestran que la duración HTTP prácticamente coincide con la del pipeline interno.

Comparación directa contra el Escenario A: el endpoint de análisis (~1.7–2.5 s) es entre 140x y 200x más lento que /health/ (p50 12.1 ms) — y en las 3 muestras, la llamada a Gemini por sí sola (1.56–2.31 s) explica entre el 87% y el 94% del tiempo total del pipeline.

6. Cuello de botella / comportamiento inesperado identificado

Cuello de botella confirmado: la llamada a Gemini domina el pipeline

Los números de la sección 5 confirman con evidencia medida (no solo teórica) el riesgo ya documentado en **riesgos-técnicos.md**: el pipeline ejecuta OCR/extracción + spaCy + Gemini **síncrono dentro del mismo request**, y Gemini (1.56–2.31 s) explica 87–94% del tiempo total — extracción de texto y spaCy juntas nunca superan los 165 ms. Con gunicorn corriendo 3 workers síncronos, en el peor caso solo 3 análisis pueden procesarse en paralelo; cualquier solicitud adicional queda en cola 1.5–2.5 segundos.

Comportamiento inesperado: latencia elevada y variable en la primera solicitud tras iniciar el contenedor

La primera solicitud medida tras `docker compose up -d --build` (tanto `/health/` vía curl como el endpoint de análisis) mostró una duración de request HTTP notablemente mayor que las siguientes — en el endpoint de análisis, la solicitud #1 tardó 4043.7 ms de request HTTP contra solo 2470.0 ms de pipeline interno (diferencia de ~1.57 s no explicada por el propio análisis); las solicitudes #2 y #3, ya "calientes", no mostraron esa diferencia. El mismo patrón apareció en las primeras llamadas a `/health/` vía curl (~1.7–1.8 s) antes de correr el script de medición, que en cambio mostró un p50 de solo 12.1 ms sobre 20 solicitudes.

Interpretación: consistente con un efecto de "arranque en frío" — conexión inicial a PostgreSQL, imports diferidos de Python en el primer request de cada worker de gunicorn, o resolución de `localhost` del lado del cliente en Windows. No se investigó la causa exacta a fondo (fuera del alcance de esta entrega), pero queda documentado: **la primera solicitud tras un despliegue o reinicio no es representativa de la latencia normal del servicio.**

7. Mejora aplicada (con comparación antes/después)

Antes: con `DEBUG=False` (configuración real del contenedor Docker), ningún log de la aplicación llegaba a ningún lado. El bug `InvalidStorageError` de Semana 4 solo pudo diagnosticarse levantando manualmente una segunda instancia del contenedor con `DEBUG=True`.

Después: `LOGGING` configurado explícitamente en `settings.py` (sección 3) — los logs se ven siempre, en cualquier ambiente, en JSON estructurado, correlacionados por `request_id`, sin depender de `DEBUG`. Evidencia real (mismo contenedor, `DEBUG=False`): las secciones "Evidencia de una solicitud exitosa" y "de una entrada inválida" arriba muestran logs completos, visibles con un simple `docker compose logs web`.

Bonus de esta entrega: instrumentar el middleware reveló un **segundo bug real** — la primera versión de `core/middleware.py` reseteaba la variable de contexto *antes* de emitir su propio log `request_completada`, así que ese log salía con `request_id: null` (verificado en vivo). Corregido moviendo el `logger.log(...)` a dentro del `try`, antes del `reset()` en el `finally`. Verificado de nuevo tras el fix: `request_id` correcto en todos los logs mostrados en este documento.

8. Plan de escalabilidad

¿Qué crecimiento se está considerando y cuál es la restricción observada?

Más usuarios subiendo documentos de forma concurrente. La restricción observada es que el pipeline corre síncrono dentro del mismo worker de gunicorn — el número de análisis simultáneos está limitado a 3 workers, no a la capacidad real de CPU/red disponible.

¿Qué mejora debe realizarse primero?

Mover el pipeline a una cola de tareas asíncrona (Celery o RQ + Redis) — la vista devuelve inmediatamente un `202 Accepted` con un ID de tarea, y el análisis corre en un worker separado. Es lo que más impacto tendría porque ataca directamente el cuello de botella medido.

¿Cuándo convendría usar caché, workers, colas o más instancias?

- **Cola de tareas:** en cuanto el volumen supere lo que 2-3 workers síncronos puedan procesar sin timeouts perceptibles.
- **Caché:** de resultados por hash del archivo, para no repetir la llamada a Gemini con el mismo documento.
- **Más instancias/workers:** solo tiene sentido después de mover a cola de tareas — agregar workers síncronos hoy solo movería el cuello de botella a la cuota de Gemini.

¿Qué impacto tendría en memoria, datos, privacidad y costo?

- **Memoria:** Redis como broker agrega un servicio más (RAM adicional, modesta para este volumen).
- **Datos/privacidad:** la cola transportaría temporalmente el texto extraído (dato potencialmente sensible) entre proceso web y worker — requiere TTL corto y borrado tras procesar.
- **Costo:** un worker adicional corriendo permanentemente tiene costo incluso sin carga; en los free tiers evaluados en Semana 4 esto probablemente ya no entra gratis.

¿Qué indicador permitiría decidir cuándo escalar?

El p95 de duración del endpoint principal sostenido por encima de un umbral (ej. 8-10s), o tasa de error/timeout creciente, o consumo de cuota diaria de Gemini acercándose al límite (métrica pendiente).

9. Limitaciones pendientes

- No se implementó la cola de tareas (Celery/RQ) en esta entrega — documentado como siguiente paso, coherente con "no es obligatorio implementar infraestructura adicional" del descriptor.
- No hay métrica de consumo de cuota de Gemini todavía.
- El CI no corre `scripts/medir_rendimiento.py` automáticamente — la medición fue manual.
- Los logs se escriben a `stdout` del contenedor pero no se envían a ningún sistema centralizado de agregación (Loki, ELK) — suficiente para el alcance académico actual.

Repositorio actualizado

<https://github.com/DanCastSV/TributIA>

Incluye código fuente actualizado, middleware e instrumentación (`core/middleware.py` , `core/logging_context.py` , `core/logging_utils.py`), script de medición (`scripts/medir_rendimiento.py`), README con instrucciones para ejecutar la medición, `.env.example` sin claves reales, resultados en `docs/` (`docs/observabilidad-semana5.md` , `docs/medicion_health.json`), y pruebas/pipeline/despliegue de semanas anteriores.